AI1220 — Software, Web & Mobile Engineering: Code without Limits

Assignment 1: Requirements Engineering,

Architecture & Proof of Concept

Campus Social Network

Student Name	Student ID
Ali Yaroobi	25011226
Saidamirkhon Yusupov	25011163
Ahmed Waqqas	25011081
Rawda Alkhoori	25011159

September 2026

Contents

Product Scope Statement 4

Section 1: Requirements Engineering (30%) 5

1.1 Stakeholders & Power-Interest Analysis 5

Stakeholder Conflict Identification & Design Resolutions 5

1.2 Functional Requirements (20 FRs) 7

Identity & Access Management 7

Communities & Spaces 7

Publishing & Content Creation 7

Feed Filtering & Discovery 7

Audience Scoping 7

Collaborative Drafting 8

Engagement & Interactions 8

Content Moderation & Appeals 8

Campus Notifications 8

AI Assistance (PulseDigest) 8

Technology Constraints 9

1.3 Non-Functional Requirements (Measurable Targets) 9

1.4 User Stories & Acceptance Criteria 10

1.5 Validation & Traceability Matrix 12

Section 2: System Architecture (45%) 13

2.1 Ranked Architectural Drivers 13

2.2 System Design via C4 Model 14

C4 Level 1: System Context Diagram (docs/diagrams/System Context Diagram.drawio) 14

C4 Level 2: Container Diagram (docs/diagrams/Container Diagram.drawio) 15

C4 Level 3: Backend Component Diagram (docs/diagrams/Backend Component Diagram.drawio) 16

2.3 Behaviour & Component Responsibilities 16

2.4 Interface Contracts 18

1. POST /api/auth/token 18

2. GET /api/posts 19

3. GET /api/posts/{id} 19
4. POST /api/posts 19
5. POST /api/posts/ai/preview (Circuit Breaker Protected) 20
6. Community Spaces (GET & POST /api/communities) 20
7. Engagement Endpoints (Comments & RSVPs) 20
8. Moderation Endpoints 20
9. Standard Common Error Schema 20
10. WebSocket Protocol & Event Catalog (/ws/collaborate/{draft_id}) 21
2.5 Code & Repository Structure 21
2.6 Relational Data Model (Entity Relationship Diagram) 22
2.7 Architecture Decision Records (ADRs) 24
ADR-01: Modular Monolith vs. Microservices Architecture 24
ADR-02: Optimistic Concurrency Control vs. CRDTs for Collaborative Drafting 25
ADR-03: Application-Layer Query Scoping with PostRepository vs. PostgreSQL Row Level Security (RLS) for Audience Privacy 25
ADR-04: Isolated Circuit-Breaker with Heuristic Fallback for AI Services 26
Section 3: Project Management and Team Collaboration (15%) 26
3.1 Team Structure & Responsibilities 26
Team Coordination Practices 26
3.2 Development Workflow & AI Policy 27
3.3 Development Approach & Methodology 27
3.4 Risk Analysis & Mitigation Matrix 28
3.5 Assignment 2 Timeline & Verifiable Milestones 28
Section 4: Proof of Concept Summary (10%) 29
4.1 Scope & Architectural Simplifications 29
4.2 Contract Verification 29
4.3 Setup & Run Instructions 30
4.4 Three-Minute Demonstration Script 31
Section 5: References 32

Product Scope Statement

UniPulse establishes an integrated campus social and communications platform designed to eliminate campus communication fragmentation. To guarantee predictable delivery and sound academic software engineering, project boundaries are partitioned across three distinct planning horizons:

1. In-Scope for Release 1 (Core Foundation & PoC):

• Mock user authentication supporting User ID, university email, role (Student, Club Executive, Faculty, Moderator), and department claims with cryptographically signed session tokens.
• Rich-text announcement authoring with title bounds (3–120 characters), body bounds (5–5,000 characters), Markdown formatting, and event metadata.
• Multi-tier audience scoping (Public, Campus Only, Club Members, Faculty Only) enforced strictly at the backend query layer.
• Community space creation, discovery, and dynamic membership management (admin and member roles).
• Real-time collaborative co-drafting over persistent WebSockets with monotonic integer versioning and optimistic concurrency control.
• PulseDigest AI event extraction and summarization workflow operating strictly under the human-in-the-loop principle ('AI drafts, human verifies').
• Community content moderation with automated 3-flag quarantine triage and author appeal submission.

2. Deferred to Subsequent Releases (Assignment 2):

• Direct integration with enterprise campus LDAP / Active Directory and SAML/OIDC federated identity providers.
• Multi-signature moderation consensus requiring two independent moderator approvals for appeal reinstatements.
• Distributed Redis Pub/Sub cluster mode for multi-node WebSocket connection fan-out across server nodes.
• Peer-to-peer private direct messaging and binary media/video upload streaming with CDN caching.

3. Out of Scope (Will Not Be Built):

• Commercial monetization, advertising tracking, sponsored post bidding, or commercial affiliate integration.
• Open public self-registration for non-university external entities without verified institutional credentials.
• End-to-end encrypted peer messaging beyond institutional compliance oversight.
• Fully autonomous AI content censorship without human moderator review and appeal channels.

Section 1: Requirements Engineering (30%)

1.1 Stakeholders & Power-Interest Analysis

Software engineering begins by understanding every party the system touches [1]. As emphasized in Lecture 02 of Week 02, a stakeholder is anyone affected by the system, not merely its direct end users [1], [2]. Four distinct stakeholder groups have been identified and mapped across a Power vs. Interest matrix:

1. Student Club Executives (Key Players – High Power, High Interest): Goals: Promote society workshops, co-author announcements, broadcast campus events, and track attendee engagement. Concerns: Insufficient audience reach, fragmented communication tooling, and audience notification fatigue. Influence: Primary content creators driving daily platform interaction and engagement.

2. Undergraduate & Graduate Students (Keep Informed – Low Power, High Interest): Goals: Discover timely, relevant campus events, participate in academic discussions, and control visibility of personal vs. academic updates. Concerns: Notification spam, information overload, privacy exposure, and toxic peer interactions. Justification for Low Power: While students constitute the vast majority of end-users (~90% of traffic), individual students lack institutional policy authority or veto power over campus IT systems, correctly placing them in the 'Keep Informed' quadrant under standard strategic stakeholder analysis [2].

3. University Administration & Faculty (Keep Satisfied – High Power, Low Interest): Goals: Broadcast official campus safety notices, academic deadlines, and maintain institutional reputation and policy compliance. Concerns: Unmoderated hate speech, academic dishonesty, and unauthorized university announcements. Influence: Regulatory authority, legal liability oversight, and institutional veto power.

4. Campus Safety & Content Moderators (Key Players – Medium Power, High Interest): Goals: Rapidly triage reported violations, uphold community standards, review author appeals, and maintain auditable moderation logs. Concerns: Tooling deficiencies, alert volume bottlenecks, and ambiguous escalation paths. Influence: Operational policy enforcement and regulatory compliance.

Stakeholder Conflict Identification & Design Resolutions

Conflict 1 (Reach vs. Noise): Student club leaders demand broad, campus-wide broadcast reach to maximize workshop turnout, whereas general students demand strict feed filtering to prevent announcement spam.
Design Resolution: UniPulse resolves this by introducing role-gated audience scopes (Public, Campus Only, Club Members, Faculty Only) at the database query layer, paired with customizable student feed interest filters.

Conflict 2 (Speed vs. Safety): Club executives desire instantaneous unvetted post publication to announce last-minute room changes, whereas university administrators require prior review to

ensure safety and policy compliance.
Design Resolution: UniPulse employs reactive automated quarantine: posts publish immediately without pre-moderation delay, but upon receiving 3 distinct community flags, the post is automatically quarantined from feeds pending moderator review.

Conflict 3 (Informal vs. Official): General students demand open, informal peer discussion threads, whereas faculty members require dignified, verified disclosures untainted by extraneous chatter.
Design Resolution: Official administrative notices are partitioned into dedicated announcement boards with commenting restrictions, separating institutional advisories from informal campus discussion feeds.

Conflict 4 (Coordinated Flag Abuse & Brigading): Flag abuse & brigading is conflict 4 (Coordinated Flag Abuse & Brigading): This may be done by malicious groups, and/or a combination of rival student societies and/or 3 users marking an accurate announcement off on a different flag.
Design resolution: Flags will be different from verified accounts with different session fingerprints; 12 hours appeal triage period for authors; authors' reporting history will be visible in reporting history in the ring; bad faith in reporting history will result in disciplinary action and loss of flagging privileges.

Institutional Data Protection Compliance: All Student Data and audience scoping is compliant with the University Student Data Protection Policy and the UK/EU GDPR principles – never sharing private academic notices.

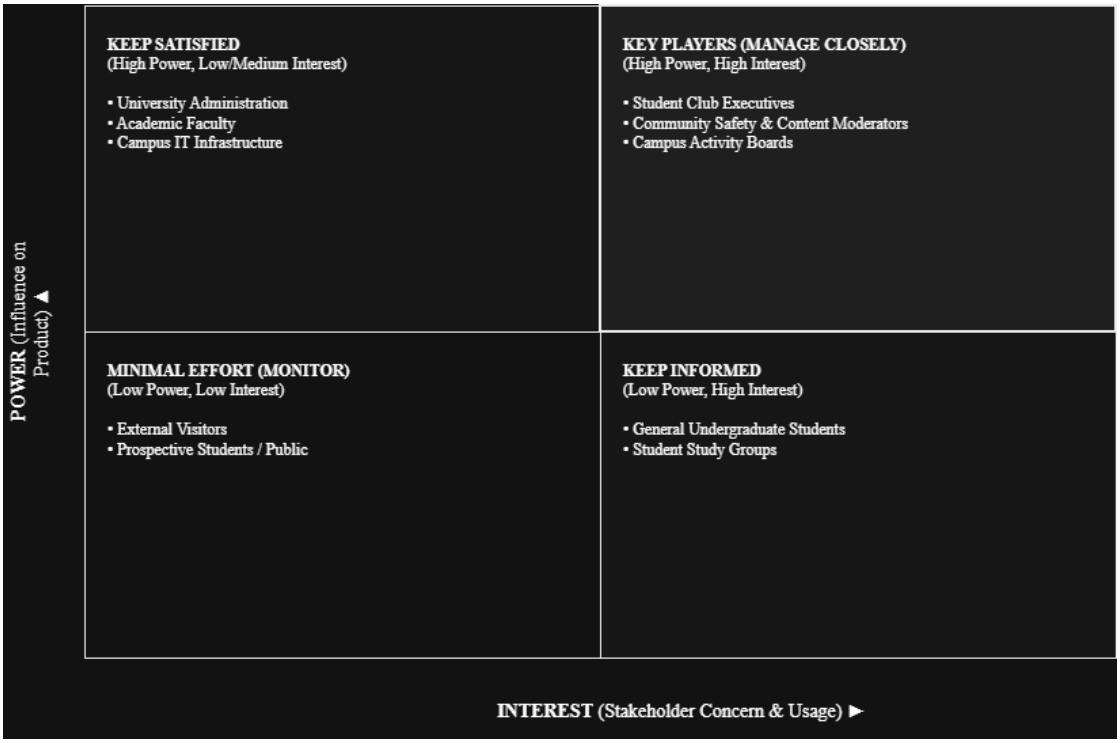


Figure 1.1: Stakeholder Power vs. Interest Grid (Mendelow Matrix)

1.2 Functional Requirements (20 FRs)

Functional requirements define observable, testable system behaviors across 9 distinct functional areas in accordance with Week 02 requirements engineering principles [1], [3]:

Identity & Access Management

FR-ID-01: The system shall authenticate users via university credentials, issuing a cryptographically signed session token containing user_id, role, and department.

FR-ID-02: The system shall validate session tokens on every protected endpoint request to establish identity without re-querying the external identity provider.

FR-ID-03: The system shall reject expired or forged session tokens, returning an HTTP 401 Unauthorized response with an explicit error code.

Communities & Spaces

FR-COMM-01: Authorized users shall be able to create structured community spaces categorized as Student Club, Academic Department, or Special Interest Group.

FR-COMM-02: The system shall allow users to search, join, and leave community spaces, dynamically updating membership records and associated audience access rights.

Publishing & Content Creation

FR-PUB-01: Authors shall create rich-text posts with titles constrained between 3 and 120 characters, and body content constrained between 5 and 5,000 characters, supporting Markdown formatting.

FR-PUB-02: Authors shall be able to assign category labels, event flags, and structured event metadata (date, time, venue) to published posts.

FR-PUB-03: Authors shall be permitted to edit previously published posts, generating an auditable version history in the datastore.

Feed Filtering & Discovery

FR-FEED-02: Students can narrow down the announcements and view only the events in a specific category and specific topic tags of interest, thus reducing the "noise" of the announcements.

Audience Scoping

FR-AUD-01: Authors shall designate a mandatory audience tier for every post: Public, Campus Only, Club Members, or Faculty Only.

FR-AUD-02: The system shall filter feed queries strictly according to the authenticated user's verified role and memberships; unauthorized direct access requests via GET /api/posts/{id} shall return HTTP 403 Forbidden.

Collaborative Drafting

FR-COL-01: Multiple authorized co-editors shall establish persistent WebSocket connections to co-author draft announcements in real time.

FR-COL-02: The system shall use optimistic concurrency control (OCC) for draft changes, combined with monotonic integer versions checking for every draft patch payload that comes in.

FR-COL-03: If the base version of the patch is stale, the server shall not accept the patch with a VERSION_OUTDATED WebSocket error event message, and shall broadcast the version of the server document.

Engagement & Interactions

FR-ENG-01: Authenticated users shall post comments constrained between 1 and 1,000 characters on authorized announcements.

FR-ENG-02: Authenticated users shall submit RSVPs (Attending, Maybe, Declined) to event-flagged announcements, updating event attendee counts.

FR-ENG-03: Community moderators shall be able to remove objectionable comments, recording author notification and reason in the audit log.

Content Moderation & Appeals

FR-MOD-01: Authenticated users shall flag objectionable posts, submitting a mandatory categorization reason (Spam, Harassment, Policy Violation).

FR-MOD-02: Upon receiving 3 distinct user flags, the system shall automatically set post status to QUARANTINED, hide the post from public feeds, and route it to the moderator triage queue.

FR-MOD-03: Authors of quarantined posts shall be entitled to submit an appeal with written justification within 7 days; moderator resolutions shall append an immutable record to the audit log.

FR-MOD-04: Campus moderators have the following powers: They can remove posts, set flags as forfeited, restore flags (if there are flags which are not valid), and temporarily suspend abusive accounts.

Campus Notifications

FR-NOTIF-01: The system will automatically email notifications to the recipients for the "quarantine" changes, when the appeal is successful and when the event schedule changes.

AI Assistance (PulseDigest)

FR-AI-01: The system shall provide an automated event extraction and summarization assistant (PulseDigest) that extracts structured event metadata (date, time, venue) and produces a 2-sentence summary.

FR-AI-02: Following the 'AI drafts, human verifies' principle [4], AI summaries and extracted metadata shall be returned to the author as editable draft proposals and shall not be persisted to the database without explicit author confirmation.

PoC Identity Simplification Note: Note: For Release 1 PoC, the author_id is included in both the request header and in this case the mock roles are specified as a simplification for testing a multi-role contract using non-live campus LDAP servers, intentionally and documented.

Technology Constraints

• Backend Framework: Python 3.11+ using FastAPI for asynchronous ASGI concurrency and native WebSocket handling.
• Frontend Rich Text: Single Page Application utilizing Tiptap (ProseMirror engine) for Markdown serialization.
• AI Integration: External LLM requests encapsulated behind a 3.0-second timeout with deterministic heuristic fallback.

1.3 Non-Functional Requirements (Measurable Targets)

Non-functional requirements specify operational qualities, constraints, and performance metrics, each backed by an explicit rationale [1]:

NFR-PERF-01 / NFR-SCAL-01 (Latency & Capacity Standards): The default REST feed & post endpoint has criteria of p95 < 200ms @ under 200 req/s, the system capacity has criteria of 1000 active concurrent users with p95 < 300ms and the AI preview generation has a separately defined SLA of 3.0s circuit-breaker. Rationale: Harmonizes peak browsing responsiveness between lectures with realistic circuit breaker allowances for external AI APIs.

NFR-AVAIL-01 (Availability): Core feed, publishing, and authentication services shall maintain 99.9% uptime across the 15-week academic semester, excluding scheduled maintenance. Rationale: Unplanned outages during exam periods or student elections severely undermine campus operational trust.

NFR-RESIL-01 (Resilience & Recovery): Container crashes shall trigger automated restart and health-probe recovery within 15 seconds (RTO <= 15s), with a Recovery Point Objective of zero data loss (RPO = 0s) for all committed database transactions. Rationale: Protects published announcements, draft versions, and event RSVPs from data corruption during infrastructure restarts.

NFR-SEC-01 (Security & Identity): All network endpoints shall enforce TLS 1.3 encryption; no hardcoded secrets or keys shall be committed to version control; university SSO serves as production identity, while local developer mock authentication hashes passwords using Argon2id [5]. Rationale: Eliminates credential exposure risks over unencrypted campus Wi-Fi networks and prevents secret leaks in shared repositories.

NFR-PRIV-01 (Data Privacy & AI Minimization): Student prompts are not transmitted to external LLM services – only sanitized title and body text are sent, the retention for external

LLM API data will be set to 0 days, and deletion requests will delete all drafts within 30 days. Justification: Students will be protected from infringement of institutional privacy and protect student intellectual property.

NFR-USAB-01 (Accessibility): The browser interface shall fully comply with WCAG 2.1 Level AA standards [6], including full keyboard navigation and minimum 4.5:1 text contrast ratios. Rationale: Fulfills institutional legal mandates ensuring universal access for students with visual or motor impairments.

NFR-USAB-02 (Usability Benchmark): Each new student club author can write, scope and publish an announcement with <60 seconds with 3 clicks. The reason for this is that the cognitive load is low and it is seamlessly integrated into students' societies.

NFR-AUDIT-01 (Auditability & Retention): All moderation actions (quarantines, dismissals, appeals) shall append immutable audit log records retained for 365 days, containing timestamp, actor ID, action type, and justification. Rationale: Provides legal defensibility and evidence retention under University Student Data Protection Policy and disciplinary procedures.

1.4 User Stories & Acceptance Criteria

User stories represent core stakeholder requirements across normal workflows, system failures, and policy exceptions, formatted in Given/When/Then acceptance syntax [7], [8]:

User Story US-01 (Club Executive - Collaborative Announcement):
As a student club vice-president, I want to co-author an event announcement with the club secretary in real time, so that we can verify room bookings and publish an accurate campus schedule.

• Acceptance Criteria US-01.1 (Real-Time Synchronization): Given User A and User B possess co-editor permissions on draft #42, When User A modifies the event title, Then User B's interface shall update within 500 ms via WebSocket.

• Acceptance Criteria US-01.2 (Optimistic Concurrency Conflict): If User A and User B both examine version 3 of draft #42, and User A commits version 4, and User B subsequently submits a patch based on version 3, Then the server MUST reject User B's patch, send a VERSION_OUTDATED WebSocket error event to User B and provide the latest draft state.

User Story US-02 (Faculty Department Chair - Scoped Notice):
As an academic department chair, I want to publish internal curriculum grading guidelines visible exclusively to faculty members, so that confidential academic policies are not disclosed to the public or students.

• Acceptance Criteria US-02.1 (Audience Scoping Enforcement): Given an authenticated user possessing the role 'Student', When they attempt to access GET /api/posts/{id} for a post scoped as 'Faculty Only', Then the server shall return HTTP 403 Forbidden with error code FORBIDDEN_AUDIENCE.

User Story US-03 (Content Moderator - Toxic Content Triage):
As a campus content moderator, I want abusive posts to be quarantined automatically once

reported by multiple students, so that community safety guidelines are upheld without manual pre-moderation delays.

• Acceptance Criteria US-03.1 (Automated Threshold Quarantine): Given an active public post has accumulated 2 policy flags, When a third unique authenticated user submits a flag, Then the post status shall immediately transition to QUARANTINED, the post shall be omitted from feeds, and a triage ticket shall be generated.

User Story US-04 (General Student Reader - Feed Summary Display):

As a general student reader, I want to have a quick 2 sentence summary and event badge in my campus feed, then see if I want to read through the lengthy promotional text to see if I want to attend.

• Acceptance Criteria US-04.1 (Feed Summary Display): If an announcement has an approved AI summary and event metadata, then when the student scrolls past the card in the feed, the 2-sentence summary, date badge and venue shall be prominently displayed.

User Story US-05 (Announcement Author - Review & Circuit Breaker):

As the announce author, I would like to review the AI-generated event date, venue and summary before publishing, to make sure it doesn't contain inaccuracies or the AI generates an accurate summary.

• Acceptance Criteria US-05.1 (Author Review & Explicit Confirmation): Then, the system will provide the user with an editable 2-sentence-summary plus the event metadata that it has extracted from the text, within 3.0 seconds, the user shall explicitly click "Accept Summary" to store this summary into the system database after announcing the text.

• Acceptance Criteria US-05.2 (Circuit Breaker Resilience): If when an author requests an AI preview, the external LLM provider latency is greater than 3.0 seconds, or if the external LLM provider returns an HTTP 5xx response code, Then the circuit breaker shall trip, generate a deterministic local extractive summary, return an HTTP 200 OK response with the is_fallback field set to true, and alert the author that they are in fallback mode.

User Story US-06 (Author Appeal for Quarantined Post - Policy Flow):

As a student club author whose announcement was incorrectly quarantined, I want to submit an appeal with supporting justification, so that a human moderator can review and reinstate my post.

• Acceptance Criteria US-06.1 (Appeal Submission): If the author has requested the announcement be set to QUARANTINED status and included an appeal reason in the appeal window within 7 days of the announcement, then the status of the entity shall be changed to PENDING and a notification shall be sent to the moderator queue and an immutable entry shall be logged in AUDIT_LOG.

• Acceptance Criteria US-06.2 (Moderator Reinstatement): If an appeal is in PENDING status, when a Campus Moderator reviews and approves the appeal, then the post's status shall change to PUBLISHED and the post shall reappear in the public feed.

1.5 Validation & Traceability Matrix

Requirements validation followed five core criteria: validity, consistency, completeness, realism, and verifiability [1]. Peer walkthroughs identified and fixed two notable contradictions:
1. Moderation Threshold: An initial drafting note referenced a 'word count threshold' for moderation; this was formally rectified to require exactly 3 unique user flags, aligning functional requirements with US-03 and Section 2.3.
2. Publishing Length Bounds: An inverted character constraint was corrected so that titles are bounded to 3–120 characters and body content is bounded to 5–5,000 characters, consistent with backend Pydantic schemas.

Need ID	Stakeholder Need	Functional Req	Non-Functional Req	User Story	Architecture Component
N-01	Event Visibility & Reach	FR-PUB-01, FR-PUB-02, FR-FEED-02	NFR-PERF-01	US-01, US-04	PostRouter, PostRepository
N-02	Real-Time Co-Authoring	FR-COL-01, FR-COL-02, FR-COL-03	NFR-PERF-01	US-01	WebSocketRouter, RedisBroker
N-03	Role-Based Privacy Scoping	FR-AUD-01, FR-AUD-02	NFR-SEC-01, NFR-PRIV-01	US-02	AudienceGuardMiddleware
N-04	Abuse & Safety Triage	FR-MOD-01, FR-MOD-02, FR-MOD-03, FR-MOD-04	NFR-AUDIT-01	US-03, US-06	ModerationRouter, AuditService
N-05	Info Overload Reduction	FR-AI-01, FR-AI-02	NFR-RESIL-01, NFR-USAB-02	US-04, US-05	PulseDigestAIService
N-06	Institutional Auditing	FR-ID-01, FR-ID-03	NFR-AUDIT-01	US-02, US-06	AuthRouter, AuditLogRepository

Assumptions:
• 100% of campus members possess an active institutional directory ID and email account.
• Peak concurrent active user load for Release 1 is capped at 1,000 simultaneous users.
• External LLM requests will not exceed 5,000 characters per single summarization call.
• Campus moderators commit to reviewing queued appeals within a 24-hour service level target.

Open Questions:
• Should audit log records be permanently archived to cold storage after 365 days or permanently purged to minimize data liability under University Data Protection guidelines?
• Should appeals for hate-speech quarantine require two independent moderator signatures before reinstatement? (Scheduled for resolution in Assignment 2).

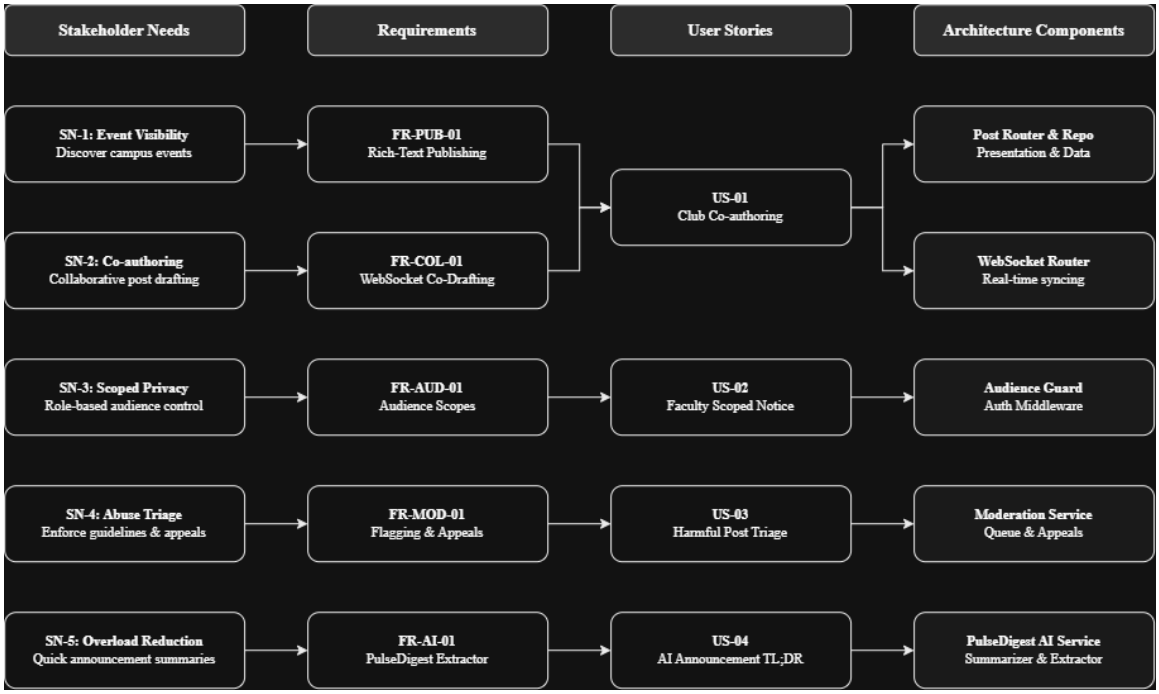


Figure 1.2: Requirements Traceability Flow (Needs to Components)

Section 2: System Architecture (45%)

2.1 Ranked Architectural Drivers

Architectural drivers represent the prioritized quality attributes, functional demands, and technical constraints that govern architectural decision-making (Week 04) [9]. For UniPulse, these drivers are ranked in explicit priority order:

1. Strict Audience Authorization (Driver 1 – Highest Priority; FR-AUD-01, NFR-SEC-01, NFR-PRIV-01): Exclusively authorized access to sensitive announcements (e.g., faculty-only grading policies, private club drafts) must be enforced deterministically at the backend query layer. Unlike consumer social networks, campus privacy violations carry severe University Student Data Protection Policy and UK/EU GDPR compliance penalties. Driver 1 outranks all other concerns because privacy leakage cannot be remedied post-hoc.

2. Real-Time Collaboration Performance (Driver 2; FR-COL-01, NFR-PERF-01): Synchronized multi-editor drafting requires sub-500 ms propagation across active co-authors. This necessitates native WebSocket duplex channels and lightweight optimistic concurrency versioning. Driver 2 outranks general throughput because real-time co-authoring is UniPulse's core differentiator over static broadcast boards.

3. Modularity & Independent Testability (Driver 3; NFR-SCAL-01, NFR-RESIL-01): As taught by Fabio Pizzati in Week 04, maintainable software requires strict separation of concerns to avoid 'software rot' [10]. UniPulse adopts a Modular Monolith structure to preserve clean

subsystem domain boundaries without the severe network latency, distributed transaction complexity, and operational overhead of microservices.

4. AI Resilience & Decoupled Latency (Driver 4; FR-AI-01, NFR-RESIL-01): External LLM APIs exhibit variable response latencies (3–10 seconds) and sporadic rate-limit exhaustion. The core publishing and browsing pipeline must be fully decoupled from the AI assistant via timeouts and circuit breakers, ensuring external AI failures never degrade core campus communication.

2.2 System Design via C4 Model

The system architecture is visualised using the C4 model (Context, Container, Component, Code) introduced in Week 04 [11]. Diagram source models are maintained in editable Draw.io format:

C4 Level 1: System Context Diagram (docs/diagrams/System Context Diagram.drawio)

UniPulse sits at the center of the campus digital communications ecosystem. Primary actors comprise General Students, Student Club Executives, Faculty Members, and Campus Safety / Content Moderators. The system boundary interfaces with three external systems: (1) University SSO / LDAP for user identity authentication; (2) External LLM Provider API for summarization; and (3) Campus SMTP Mail Server for notification alerts.

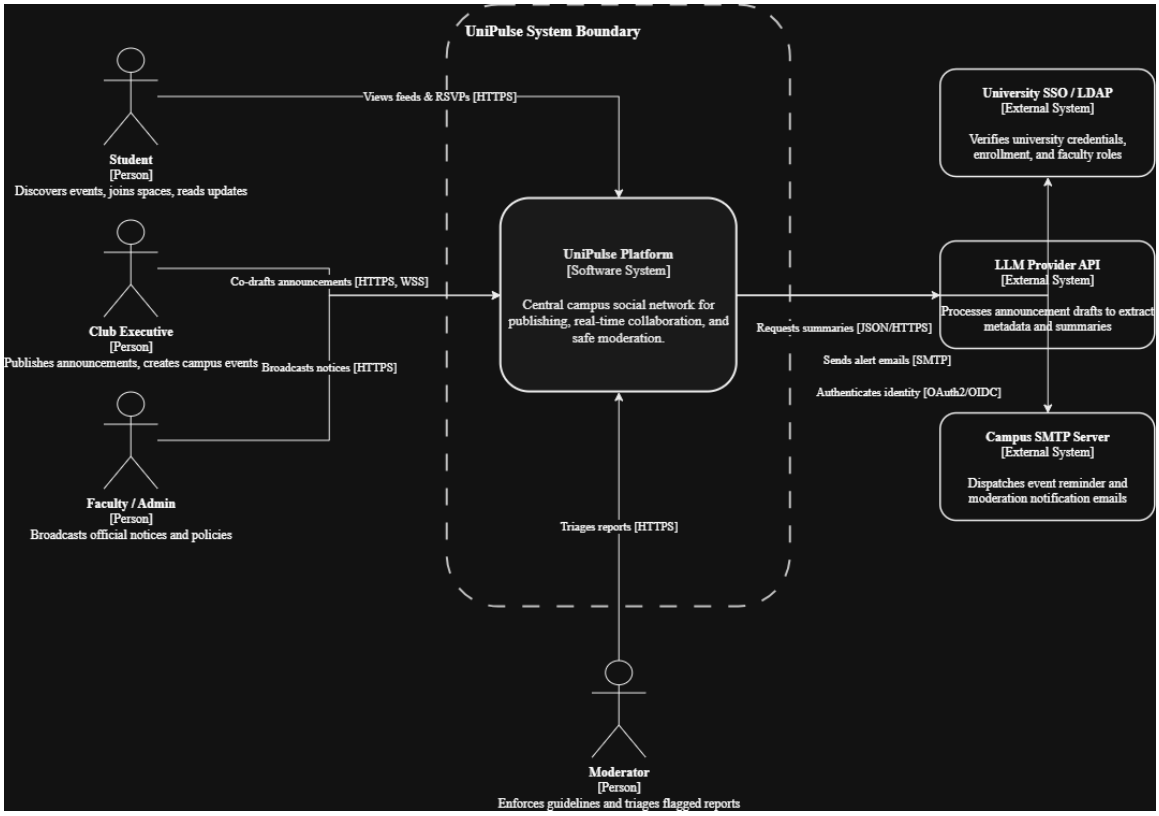


Figure 2.1: C4 Level 1 - System Context Diagram

C4 Level 2: Container Diagram (docs/diagrams/Container Diagram.drawio)

UniPulse decomposes into four cohesive containers:

1. Single Page Application (SPA): Built with React 19 and TypeScript, running in the user's browser. Employs the Tiptap rich-text editor for Markdown authoring and renders real-time feed updates.
2. Backend API Application: Asynchronous Python 3.11+ application using the FastAPI framework. Exposes RESTful JSON endpoints and WebSocket duplex channels, enforcing domain logic and authorization.
3. Relational Database: PostgreSQL instance providing ACID transactional persistence, relational constraints, foreign keys, and indexed query scoping for users, posts, memberships, and audit logs.
4. In-Memory Store & Message Broker: Redis instance utilized for high-throughput query caching and pub/sub message distribution across WebSocket connections.

Technology Selection Rationales:

• Why FastAPI: Provides asynchronous ASGI concurrency, native WebSocket support, automatic OpenAPI contract generation, and Pydantic data validation in Python.
• Why PostgreSQL: Delivers rock-solid ACID transactions, relational integrity constraints, composite indexing on (audience, community_id), and append-only audit trail reliability.
• Why Redis: Offers sub-millisecond pub/sub message broker capabilities essential for horizontal WebSocket event fan-out, alongside TTL-based caching for feed queries.
• Why React & Tiptap: React 19 enables a reactive component model, while Tiptap (built on ProseMirror) provides headless, extensible rich-text authoring that serializes cleanly to Markdown.

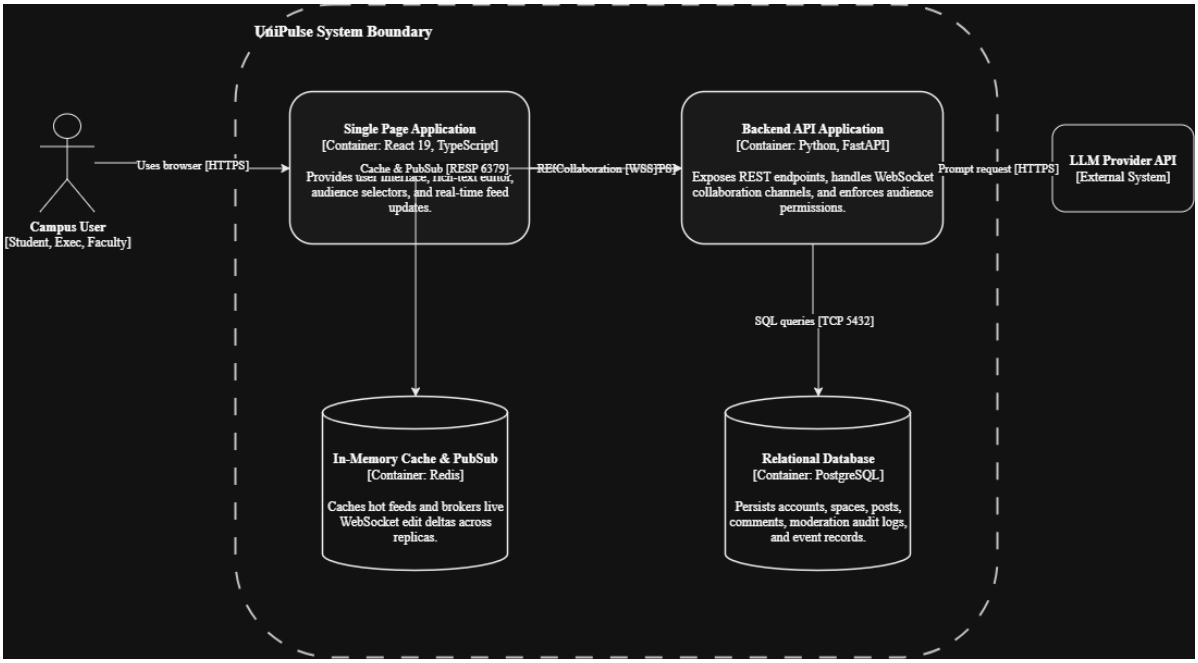


Figure 2.2: C4 Level 2 - Container Diagram

C4 Level 3: Backend Component Diagram (docs/diagrams/Backend Component Diagram.drawio)

Within the FastAPI Backend Application container, responsibilities follow a clean 3-tier modular layered architecture:

1. Routers (AuthRouter, PostRouter, WebSocketRouter, ModerationRouter): Parse HTTP requests, validate incoming schemas, and route WebSocket connections.
2. Audience Guard Middleware: Intercepts requests, validates signed session tokens, and injects verified identity claims (user_id, role, department).
3. Domain Services (PostService, PulseDigestAIService, ModerationService): Encapsulate core business logic, optimistic concurrency checks, circuit-breaker execution, and moderation workflows.
4. Repositories (PostRepository, UserRepository, AuditLogRepository): Abstract database interactions, enforcing query-level audience scoping and transactional persistence.

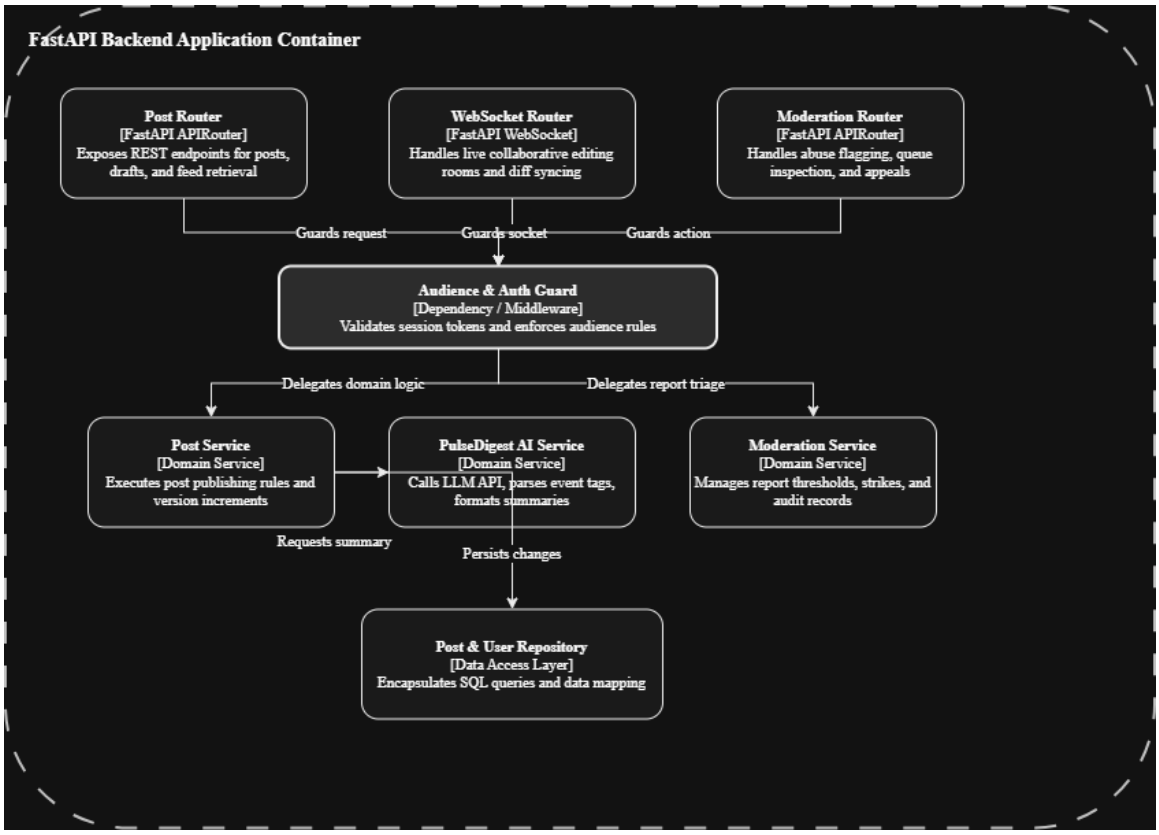


Figure 2.3: C4 Level 3 - Backend Component Diagram

2.3 Behaviour & Component Responsibilities

Subsystem Responsibilities:

• AudienceGuardMiddleware: Owns identity extraction, token signature verification, and role-based endpoint gating (401/403).

• PostService: Owns business invariants for post creation, character length validation, Markdown

sanitization, and edit lifecycle management.
• PostRepository: Owns database query parameterization, ensuring posts outside the user's audience scope are never retrieved from disk.
• WebSocketRouter & ConnectionManager: Owns active client connection pools, ping/pong heartbeats, and real-time draft patch broadcast.
• PulseDigestAIService: Owns LLM prompt construction, 3.0-second timeout enforcement, circuit breaker trip states, and deterministic fallback summarization.
• ModerationService: Owns community flag accumulation, automated 3-flag quarantine transitions, and appeal state machines.
• AuditLogRepository: Owns append-only logging of moderation actions and security events.

Draft-to-Published Lifecycle: All posts start as DRAFT. The draft can be optimistically concurred edited by the authorized co-editors of the document using WebSockets. Only the author (author_id) or community admin can make the final change to READY TO PUBLISH. There is a notification event that is dynamically published over Redis Pub/Sub and sent to active client feeds without full page reload, when the post is published.

Clarifying Appeal Outcomes: If the appeal is accepted, the post will be restored to PUBLISHED and the outcome will be 'Upheld' (or 'Approved') if it is not, the outcome will be 'Rejected' and the post will not be visible in feeds, it will only be visible in the QUARANTINED section.

Testing & Verification Strategy:
• Unit Testing: Every domain service is tested in isolation using mock repositories (e.g., verifying character boundary rules, optimistic lock rejection, and fallback extraction).
• Contract Testing: Every REST and WebSocket endpoint is verified against OpenAPI schemas using pytest and httpx, testing status codes, headers, and payload structures.
• Integration Testing: End-to-end vertical slices test complete request flows from HTTP routing through middleware, services, and repository layers.

Sequence Failure Scenarios (docs/diagrams/UML Sequence Diagram - Collaborative Drafting.drawio):
1. Optimistic Lock Conflict: When User B submits a patch for draft #42 with version 3, but User A has already committed version 4, the server rejects User B's patch, transmits a VERSION_OUTDATED WebSocket error event message, and broadcasts the current version 4 state. User B's client displays the conflict dialog and incorporates User A's changes without data loss.
2. Database Save Failure: If the persistence layer experiences a disk timeout or constraint violation during a draft save, the WebSocket router catches the exception, rolls back the transaction, does NOT increment the version number, and transmits a draft.save_failed error event ('Database transaction failed. Version not bumped.') to the author, allowing immediate retry.
3. WebSocket Disconnection: If a client's network drops, the server's ping/pong heartbeat (one

ping every 15 seconds) marks the connection dead after two missed pongs (about 30 seconds), removes the stale socket from the active room pool, and notifies remaining co-authors via presence.leave. The client automatically attempts exponential backoff reconnection.



Figure 2.4: Real-Time Collaborative Drafting Sequence Diagram

2.4 Interface Contracts

Primary system interface contracts and event schemas are formally defined below:

1. POST /api/auth/token

Mock SSO authentication endpoint to obtain a JWT session token.
Request Body: { "username": "str", "role": "student | faculty | club_exec | moderator" }
Response (200 OK): { "access_token": "str", "token_type": "bearer" }
Errors: 422 Validation Error.

2. GET /api/posts

Query Parameters: audience (optional string), community_id (optional string).
Headers: Authorization: Bearer <session_token> (or X-Mock-Role for PoC)

```
Response (200 OK):
{
  "items": [
    {
      "id": "uuid",
      "title": "string (3-120 chars)",
      "content": "string (5-5000 chars)",
      "author_id": "string",
      "author_name": "string",
      "audience": "public | campus_only | club_members | faculty_only",
      "community_id": "string | null",
      "status": "draft | published | quarantined | archived",
      "tags": ["string"],
      "ai_summary": "string | null",
      "event_detail": { "is_event": bool, "event_date": "string | null",
"event_location": "string | null" },
      "created_at": "ISO 8601 string"
    }
  ],
  "total": int
}
Errors: 401 Unauthorized, 422 Unprocessable Entity.
```

3. GET /api/posts/{id}

Headers: Authorization: Bearer <session_token>
Response (200 OK): Single Post object matching the schema above.
Errors: 401 Unauthorized, 403 Forbidden (FORBIDDEN_AUDIENCE: access denied on faculty-only posts), 404 Not Found.

4. POST /api/posts

```
Request Body (JSON):
{
  "title": "string (3-120 chars)",
  "content": "string (5-5000 chars)",
  "author_id": "string",
  "author_name": "string",
  "audience": "public | campus_only | club_members | faculty_only",
  "community_id": "string | null",
  "tags": ["string"],
  "request_ai_summary": bool,
  "event_detail": { "is_event": bool, "event_date": "string | null",
"event_location": "string | null" }
}
Response (201 Created): PostResponse object.
Errors: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 422 Validation Error.
```

5. POST /api/posts/ai/preview (Circuit Breaker Protected)

Data to be summarized – inputs: the data and event metadata to be extracted. Outputs: Summary data, event metadata. As the name implies, circuit breaker will intercept the timeout (>3.0s) and act as a failover.
Header: Authorization: Bearer <session_token> is required.
Request Body: { "text": "string (10-5000 chars)" }

```
Success Response (200 OK):
{
  "summary": "string (2 sentences)",
  "detected_event": bool,
  "suggested_date": "string | null (YYYY-MM-DD)",
  "suggested_location": "string | null",
  "suggested_tags": ["string"],
  "is_fallback": bool
}
Errors: 422 Validation Error (timeouts >3.0s return 200 OK with is_fallback:
true).
```

6. Community Spaces (GET & POST /api/communities)

GET /api/communities: Returns list of active spaces.
POST /api/communities: Body: { "name": "str", "type": "club | department | interest" }. Allowed roles: club_exec, faculty (otherwise returns 403 Forbidden).

7. Engagement Endpoints (Comments & RSVPs)

• POST /api/posts/{id}/comments: Add comments (1-1000 characters). All authenticated users allowed. Returns 201 Created.
• POST /api/posts/{id}/rsvp: Add event RSVP (attending/maybe/declined). All authenticated users allowed. Returns 200 OK with attendee count.

8. Moderation Endpoints

• POST /api/posts/{id}/flags: Body: { "reason": "str" }. Returns 201 Created with { "flag_id": "str", "flag_count": int, "status": "published | quarantined" }.
• POST /api/appeals: Body: { "post_id": "str", "justification": "str" }. Returns 201 Created with { "appeal_id": "str", "status": "pending" }.
• PATCH /api/moderation/{appeal_id}: Decision: upheld | rejected. Only moderator role allowed (returns 403 Forbidden to student). Returns 200 OK with { "status": "upheld | rejected", "audit_id": "str" }.

9. Standard Common Error Schema

```
{
  "error_code": "string (e.g. UNAUTHORIZED, FORBIDDEN_AUDIENCE, NOT_FOUND,
VALIDATION_ERROR)",
  "message": "human-readable explanation of the error",
  "details": {}
}
```

10. WebSocket Protocol & Event Catalog (/ws/collaborate/{draft_id})

Handshake: Upgrade authenticated via query parameter ?token=<session_token>.

```
Client Message (presence.join):
{ "action": "presence.join", "draft_id": "str" }

Client Message (draft.update):
{ "action": "draft.update", "draft_id": "str", "base_version": int, "patch": {
"title": "str", "content": "str" } }

Server Broadcast (draft.updated):
{ "event": "draft.updated", "draft_id": "str", "version": int, "patch": {
"title": "str", "content": "str" }, "modified_by": "str" }

Server Broadcast (presence.leave):
{ "event": "presence.leave", "user_id": "str" }

Server Error (VERSION_OUTDATED):
{ "error": "VERSION_OUTDATED", "current_version": int, "message": "Draft modified
by another co-editor. Please refresh." }

Server Error (draft.save_failed):
{ "error": "draft.save_failed", "message": "Database transaction failed. Version
not bumped." }
```

2.5 Code & Repository Structure

The repository strictly adheres to professional software engineering conventions:

```
unipulse/
├── .github/
│   ├── CODEOWNERS              # Review routing per subsystem owner
│   └── workflows/ci.yml        # CI: ruff + pytest on every pull request
├── docs/                       # Technical report, ADRs, and editable Draw.io
diagrams
│   ├── Assignment_01_Report.docx
│   ├── Assignment_01_Report.pdf
│   ├── adr/                    # ADR-01 … ADR-04 as Markdown files
│   ├── screenshots/            # PoC running (Swagger UI, server log)
│   └── diagrams/               # .drawio sources; exports/ holds the PNGs
├── src/
│   ├── backend/                # FastAPI application backend
│   │   ├── config.py           # Pydantic BaseSettings, environment variables
│   │   ├── main.py             # ASGI application entry, middleware registration
│   │   ├── models.py           # Domain entities with standardized enums
│   │   ├── repository.py       # Abstract data access and server-side query
scoping
│   │   ├── schemas.py          # Pydantic request/response validation models
│   │   ├── routers/            # HTTP and WebSocket route handlers
│   │   │   ├── health.py       # Liveness and readiness probes
│   │   │   └── posts.py        # Posts, feeds, role dependency & AI preview
│   │   └── services/           # Business logic and domain services
│   │       ├── ai_service.py   # PulseDigest NLP extraction and circuit breaker
│   │       └── post_service.py# Post authoring invariants and audience rules
│   └── frontend/               # Browser Single Page Application
```

```
│       ├── index.html         # Semantic HTML5 application shell & demo role
switcher
│       ├── style.css          # Monochrome responsive stylesheet
│       └── app.js             # Client DOM controller, role headers & AI preview
card
├── tests/                     # Automated test suite (22 comprehensive tests)
│   ├── conftest.py            # Pytest fixtures and mock client configurations
│   ├── test_bva_equivalence.py# Boundary value and equivalence partitioning
tests
│   ├── test_poc_contract.py   # REST endpoint contract, 403 scoping & auth tests
│   └── test_unit_models.py    # Unit tests for domain entities, regex & breaker
├── .env.example               # Template environment configuration
├── .gitignore                 # Exclusion rules (.venv, .env, __pycache__)
├── pyproject.toml             # Project metadata, cleaned dependencies, ruff
config
├── README.md                  # System overview and setup instructions
└── uv.lock                    # Cryptographic dependency lockfile
```

Subsystem Ownership:

• Backend Lead: Owns src/backend/routers/, repository.py, and schemas.py.
• Frontend Lead: Owns src/frontend/ (app.js, style.css, index.html).
• Architecture Lead: Owns docs/diagrams/, docs/adr/, and services/ai_service.py.
• QA & DevOps Lead: Owns tests/, pyproject.toml, and CI validation.

Secrets Isolation:

All sensitive credentials, API keys, and secret tokens are loaded dynamically from environment variables (.env) via config.py using Pydantic BaseSettings. Sensitive files are strictly barred from version control via .gitignore.

2.6 Relational Data Model (Entity Relationship Diagram)

The relational data model comprises 11 core entities designed to satisfy UniPulse's functional requirements, transactional integrity, and compliance policies:

Entities & Relational Constraints:

1. USER: user_id (PK, UUID), email (UQ, VARCHAR 255), password_hash (VARCHAR 255), name (VARCHAR 100), role (ENUM: student, club_exec, faculty, moderator), dept_id (VARCHAR 50), created_at (TIMESTAMP).
2. COMMUNITY: community_id (PK, UUID), name (UQ, VARCHAR 100), type (ENUM: club, department, interest), created_by (FK -> USER.user_id), created_at (TIMESTAMP).
3. MEMBERSHIP: membership_id (PK, UUID), user_id (FK -> USER.user_id), community_id (FK -> COMMUNITY.community_id), role_in_comm (ENUM: admin, member), joined_at (TIMESTAMP). Constraint: UNIQUE(user_id, community_id).
4. POST: post_id (PK, UUID), title (VARCHAR 120), content (TEXT, 5000 chars), author_id (FK -> USER.user_id), audience (ENUM: public, campus_only, club_members, faculty_only), community_id (FK -> COMMUNITY.community_id, nullable), status (ENUM: draft, published, quarantined, archived), category (VARCHAR 50), tags (TEXT[]), event_date (TIMESTAMP, nullable), event_location (VARCHAR 200, nullable), ai_summary (TEXT, nullable), created_at

(TIMESTAMP).
5. DRAFT: draft_id (PK, UUID), post_id (FK -> POST.post_id, nullable), version (INTEGER, default 1), title (VARCHAR 120), content (TEXT), last_modified_by (FK -> USER.user_id), updated_at (TIMESTAMP). Constraint: Monotonic integer version increment for optimistic locking.
6. DRAFT_COEDITORS (New): coedit_id (PK, UUID), draft_id (FK -> DRAFT.draft_id), user_id (FK -> USER.user_id), can_edit (BOOLEAN). Constraint: UNIQUE(draft_id, user_id). Officially authorizes co-editors to participate in WebSocket collaborative editing.
7. COMMENT: comment_id (PK, UUID), post_id (FK -> POST.post_id), author_id (FK -> USER.user_id), content (VARCHAR 1000), created_at (TIMESTAMP).
8. EVENT_RSVP: rsvp_id (PK, UUID), post_id (FK -> POST.post_id), user_id (FK -> USER.user_id), status (ENUM: attending, maybe, declined), created_at (TIMESTAMP). Constraint: UNIQUE(post_id, user_id).
9. FLAG: flag_id (PK, UUID), post_id (FK -> POST.post_id), reporter_id (FK -> USER.user_id), reason (VARCHAR 255), created_at (TIMESTAMP). Constraint: UNIQUE(post_id, reporter_id) to prevent duplicate reports by the same user.
10. APPEAL: appeal_id (PK, UUID), post_id (FK -> POST.post_id), author_id (FK -> USER.user_id), justification (TEXT), status (ENUM: pending, upheld, rejected), reviewed_by (FK -> USER.user_id, nullable), reviewed_at (TIMESTAMP, nullable).
11. AUDIT_LOG: log_id (PK, UUID), timestamp (TIMESTAMP), actor_id (FK -> USER.user_id), action (VARCHAR 50), target_entity (VARCHAR 50), target_id (UUID), details (JSONB). Constraint: Append-only table with no UPDATE or DELETE permissions.

Requirements & Policy Support:

• Fast Audience Query Scoping: The combination of POST.audience, POST.community_id, and MEMBERSHIP enables indexed SQL filtering (WHERE audience = 'public' OR (audience = 'club_members' AND community_id IN (...))), guaranteeing zero data leaks.
• Optimistic Concurrency: DRAFT.version provides lockless conflict detection for WebSocket co-editing without database row lock contention.
• Community Safety: The unique constraint on FLAG(post_id, reporter_id) enforces the 3-unique-flags rule, triggering automated transition of POST.status to 'quarantined'.
• Regulatory Compliance: The append-only AUDIT_LOG table with unique log_id guarantees immutable forensic defensibility for 365 days.

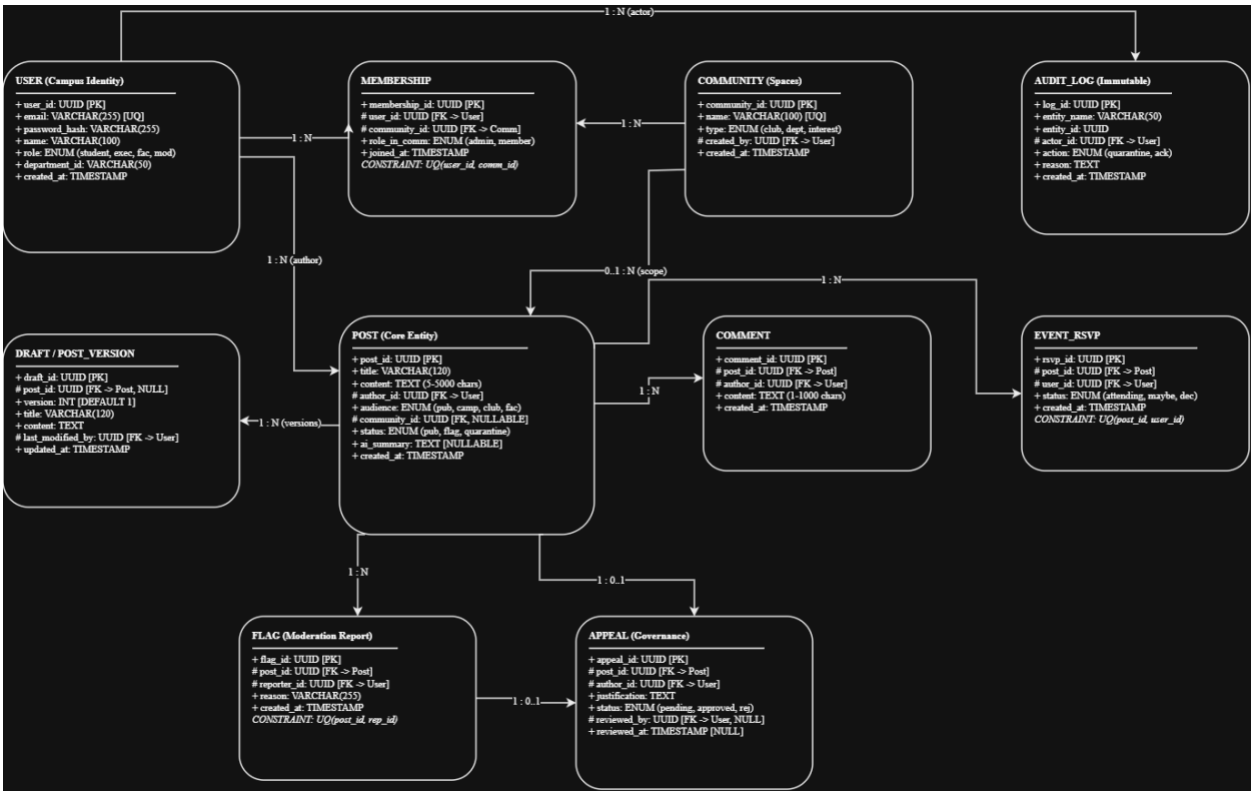


Figure 2.5: Relational Data Model (Entity Relationship Diagram)

2.7 Architecture Decision Records (ADRs)

Four formal Architecture Decision Records capture significant structural choices across different subsystems, documenting the context, options evaluated, decision made, and resulting consequences:

ADR-01: Modular Monolith vs. Microservices Architecture

• Subsystem: Core System Topology
• Context: UniPulse must support 1,000 peak concurrent users with p95 latency under 200 ms, delivered by a 4-person student team within a 15-week academic semester. Microservices introduce severe network serialization latency, distributed transaction complexity, and multi-container deployment overhead.
• Options Considered: (1) Distributed multi-container microservices; (2) Single-container modular monolith structured with strict domain-driven module boundaries.
• Decision: Adopt a Modular Monolith built on FastAPI. Subsystems (Auth, Posts, Moderation, AI) are organized as encapsulated modules with clean router-service-repository boundaries within a single process.
• Consequences:
  - Positive: Rapid zero-latency in-process function calls, shared database transactions without two-phase commit overhead, simplified single-command testing and deployment.

- Negative: Requires disciplined code reviews to prevent accidental direct coupling between internal domain services.

ADR-02: Optimistic Concurrency Control vs. CRDTs for Collaborative Drafting

• Subsystem: Real-Time Collaborative Editing (FR-COL-01)
• Context: Club executives co-draft event announcements over WebSockets. Concurrent edits must be resolved without silent overwrites or data loss.
• Options Considered: (1) Pessimistic database row locking; (2) Conflict-free Replicated Data Types (CRDTs / Yjs); (3) Optimistic Concurrency Control (OCC) using monotonic integer versioning over WebSockets.
• Decision: Implement Optimistic Concurrency Control with integer versioning. Each draft maintains a version number; incoming patches must supply the expected base version. Mismatches return VERSION_OUTDATED error event frames.
• Consequences:
- Positive: Zero database lock contention, minimal memory overhead, straightforward implementation and testing.
- Negative: In high-concurrency editing bursts, conflicting users must review the server state and re-apply their edits.

ADR-03: Application-Layer Query Scoping with PostRepository vs. PostgreSQL Row Level Security (RLS) for Audience Privacy

• Subsystem: Audience Access Control (FR-AUD-01, NFR-SEC-01, NFR-PRIV-01)
• Context: UniPulse does visibility at the role level (Public, Campus Only, Club Members, Faculty Only). A student must never receive a Faculty Only post, not even in a raw API response, so hiding content only in the user interface is not sufficient.
• Options Considered: (1) PostgreSQL Row Level Security (RLS) based on dynamic database session attributes; (2) Application-layer query scoping in PostRepository parameterized with validated JWT session claims and AudienceGuardMiddleware.
• Decision: Enforce Audience Scoping at Application Repository Layer with PostRepository, AudienceGuardMiddleware. The role of verified user and community memberships are specifically included in the SQL WHERE clause.
• Rationale & Consequences:
- Positive: The scoping rule is plain Python that can be unit-tested without a PostgreSQL instance (the PoC uses an in-memory repository); no database-specific session state is needed, so connection poolers such as PgBouncer and Redis query caching work normally.
- Negative: Protection depends on discipline: a query written outside PostRepository could bypass the filter. We mitigate this with CODEOWNERS review on repository.py and contract tests asserting 403 FORBIDDEN_AUDIENCE for students on faculty posts. A composite index on (audience, community_id) is also required to keep feed latency under 200 ms.

ADR-04: Isolated Circuit-Breaker with Heuristic Fallback for AI Services

• Subsystem: PulseDigest AI Event Extraction (FR-AI-01, NFR-RESIL-01)
• Context: PulseDigest queries external LLM APIs for summarization. External LLM services exhibit erratic response latency (3–10s) and sporadic quota exhaustion. Core post publishing must never be blocked by AI unresponsiveness.
• Options Considered: (1) Synchronous blocking LLM invocation on post creation; (2) Asynchronous isolated service with a 3.0-second timeout, circuit breaker, and deterministic local extractive heuristic fallback.
• Decision: Isolate PulseDigest behind an asynchronous circuit breaker. Invocations timeout after 3.0 seconds, tripping the breaker and immediately serving a deterministic extractive heuristic summary while notifying the author (200 OK with is_fallback: true).
• Consequences:
  - Positive: Core announcement authoring and publishing never fail due to external AI provider outages; predictable sub-second user experience.
  - Negative: When fallback is active, generated summaries rely on rule-based sentence scoring rather than full generative paraphrasing.

Section 3: Project Management and Team Collaboration (15%)

3.1 Team Structure & Responsibilities

In accordance with Brooks's Law (Week 01), expanding team size increases communication overhead according to n(n-1)/2 communication paths [15]. To eliminate bottlenecks and optimize software internal quality, our four-person team adopts clear domain ownership:

Member 1 (Backend & API Lead - Ali Yaroobi): Owns the FastAPI application core, REST routing, Pydantic data schemas, and PostgreSQL database repository layer.

Member 2 (Frontend & UI/UX Lead - Saidamirkhon Yusupov): Owns the browser Single Page App, Tiptap rich-text editor integration, DOM state management, and WCAG accessibility styling.

Member 3 (Architecture & Integration Lead - Ahmed Waqqas): Owns C4 architecture models, WebSocket duplex collaboration protocols, ERD modeling, and AI service integration.

Member 4 (QA, Requirements & DevOps Lead - Rawda Alkhoori): Owns requirements traceability, automated test suite maintenance (pytest), boundary value analysis, and Ruff linter CI enforcement.

Team Coordination Practices

• GitHub CODEOWNERS: Automatically assigns module leads to pull requests touching their domain.
• Cross-Subsystem Reviews: Every PR requires at least one review from an adjoining subsystem lead (e.g., Frontend Lead reviews Backend Post router).

• Weekly Architecture Sync: A 45-minute weekly standup aligns domain interfaces and rehearses oral defense topics, ensuring all 4 members can defend any system area.

3.2 Development Workflow & AI Policy

Git & GitHub Workflow: The team operates on trunk-based feature branching. Developers create short-lived feature branches (feat/<issue-id>-<slug>). Merging into the main branch requires 1 peer review approval and passing automated CI checks (zero Ruff lint errors and 100% passing pytest suite), defined in .github/workflows/ci.yml. Review routing is defined in .github/CODEOWNERS, and branch protection on main blocks merging until both conditions are met.

Decision Tracking: All major architectural changes are captured as formal Architecture Decision Records (ADRs) stored in docs/adr/ and committed alongside code modifications.

Rigorous AI Usage Policy ('AI Drafts, Human Verifies'): In accordance with Lecture 02 of Week 05 ('AI Practices for Software Engineering') [4], AI tools may assist in drafting requirements, boilerplate code, or test skeletons. However, AI cannot own the evidence. Every AI-assisted artifact must be rigorously reviewed by a human engineer and verified against independent tests created without shared assumptions. Plausible-but-incorrect code and hallucinated APIs are strictly guarded against.

3.3 Development Approach & Methodology

The project follows Scrum (Week 02 Lecture 01) [16] structured into three 2-week sprints across Weeks 9–14:
• Sprint 1 (Weeks 9–10): Foundation & Core Persistence: Mock SSO authentication, Role based tokens, Communities space CREATED, READ, DELETED, Post REST CRUD, baseline automated pytest suite. Definition of Done: All endpoints pass schema validation and >85% branch coverage in all domain models.
• Sprint 2 (Weeks 11–12): Rich Interaction & Collaborative Publishing: Optimistic locking, WebSocket co-editing server, Comments, RSVPs, and Campus SMTP notification hook for React Tiptap frontend. Definition of Done: 1) Two browser windows continue to update edits within 500ms, 2) For faculty audience query scoping returns 403 for student tokens in faculty posts.
• Sprint 3 (Weeks 13–14): Hardening, Moderation & AI Resilience: PulseDigest AI circuit breaker with 3.0s fallback, 3-flag quarantine triage workflow, author appeal submission, and performance benchmarking. Definition of Done: No Ruff linter warnings, Team rehearsal of questions for oral defense with p95 latency of < 200ms.
Each sprint maintains an explicit backlog prioritized via the MoSCoW method [17]. Testing and documentation are continuous first-class sprint activities rather than deferred tasks.

3.4 Risk Analysis & Mitigation Matrix

Specific project risks, their likelihood, impact, and mitigation strategies are summarized in the risk matrix below:

Risk ID	Risk Description	Likelihood	Impact	Mitigation Strategy	Contingency Response
R-01	WebSocket Connection Instability	Medium	High	Implement automatic client reconnection with exponential backoff and 15s ping/pong heartbeat.	Provide manual draft save button and fallback to HTTP polling.
R-02	External LLM API Latency & Quota Errors	High	Medium	Isolate AI calls behind a 3.0s circuit breaker; cache frequent summaries in Redis.	Fall back to deterministic extractive heuristic summarizer.
R-03	Audience Scoping Authorization Data Leak	Low	Critical	Enforce query-level filtering in database repository layer; automate RBAC contract tests.	Immediate endpoint quarantine and security audit log inspection.
R-04	Sprint Schedule Slippage	Medium	High	Enforce strict MoSCoW backlog ranking; timebox non-critical features during sprint planning.	De-scope 'Could Have' items to protect delivery of core milestones.
R-05	Team Member Illness / Absence	Medium	Medium	Maintain shared documentation, cross-subsystem PR reviews, and pair programming on critical paths.	Reallocate paired domain lead to cover critical subsystem tasks.

3.5 Assignment 2 Timeline & Verifiable Milestones

Progress toward the final submission is governed by demonstrable, verifiable milestones:

Milestone 1 (Week 10 - Core API & Data Persistence): Deploy working FastAPI backend with university SSO mock, user roles, and PostgreSQL database schema migrations. Verifiable Criteria: All authenticated REST endpoints pass automated pytest suite with >=85% branch coverage.

Milestone 2 (Week 12 - Collaborative Publishing & React UI): Connect React 19 frontend with Tiptap rich-text editor and WebSocket synchronization server. Verifiable Criteria: Two concurrent browser windows demonstrate synchronized real-time drafting within 500 ms and optimistic locking conflict detection (VERSION_OUTDATED).

Milestone 3 (Week 13 - PulseDigest AI & Moderation Triage): Implement LLM circuit breaker and 3-flag quarantine triage workflow with appeal handling. Verifiable Criteria: Flagging a post 3 times automatically removes it from public feeds; AI preview falls back gracefully within 3.0s (is_fallback: true).

Milestone 4 (Week 14 - Hardening, Packaging & Oral Defense Prep): System performance benchmarking, final documentation assembly, and team presentation dry runs. Verifiable Criteria: P95 response latency under 200 ms for core REST feed endpoints; clean Ruff linting; all 4 team members prepared for oral defense examination.

Section 4: Proof of Concept Summary (10%)

4.1 Scope & Architectural Simplifications

The Proof of Concept (PoC) demonstrates a working vertical slice connecting the browser frontend to the Python FastAPI backend over HTTP REST contracts. To focus verification on core contracts as permitted by Section 4 of the assignment brief, five deliberate simplifications were implemented:

1. In-Memory Repository: An in-memory repository (PostRepository) is used in place of a full PostgreSQL container, removing external database setup friction while testing identical CRUD interfaces.
2. Local AI Fallback Service: PulseDigest uses an in-process extractive NLP heuristic that shares the schema, latency, and output contract of an external LLM, avoiding paid third-party API dependencies.
3. Plain HTML/JS UI Shell: The interface uses semantic DOM elements to execute the exact interaction flows planned for the React SPA in Assignment 2.
4. Reactive DOM Feed Polling: Real-time feed refreshes are triggered after post submissions and via polling, deferring multi-node Redis pub/sub infrastructure to Assignment 2.
5. Simplified Identity Transport: User identity claims (user_id, role, department) are passed in request headers (X-Mock-Role) and body as an intentional Proof of Concept simplification to allow multi-role automated testing without live campus SSO infrastructure.
6. Unauthenticated Post Creation: In the PoC, POST /api/posts accepts author_id and author_name from the request body and does not yet return the 401/403 responses listed in Section 2.4. In Assignment 2 the author is taken from the verified session token and AudienceGuardMiddleware enforces 401/403 on this endpoint.

4.2 Contract Verification

The PoC frontend exchanges JSON payloads strictly adhering to the schema defined in Section 2.4. Twenty-two automated tests in tests/ verify health checks, post creation (201 Created), feed listing (200 OK), audience filtering, server-side audience scoping (403 Forbidden on faculty-only posts for students), mock bearer token authorization, boundary value conditions (422 Unprocessable Entity), regex word-boundary event detection, and circuit breaker timeout

fallback [18]. All code complies with PEP 8 and passes Ruff linter checks with zero errors and zero warnings [19].

4.3 Setup & Run Instructions

Setup and execution follow standard Python engineering habits without proprietary scripts:

1. Create and activate a virtual environment:
   python -m venv .venv
   .venv\Scripts\activate # (On Unix/macOS: source .venv/bin/activate)
2. Install dependencies:
   pip install -e ".[dev]" # app + pytest, httpx, ruff
   (Alternatively: uv pip install -e ".[dev]")
3. Start the backend application server:
   python -m uvicorn backend.main:app --app-dir src --host 127.0.0.1 --port 8000 --reload
4. Access the application in a web browser:
   Open http://127.0.0.1:8000/ to view the frontend interface.
   Open http://127.0.0.1:8000/docs to explore interactive OpenAPI documentation.
5. Execute the automated test suite:
   pytest
6. Verify code quality with the Ruff linter:
   ruff check src tests

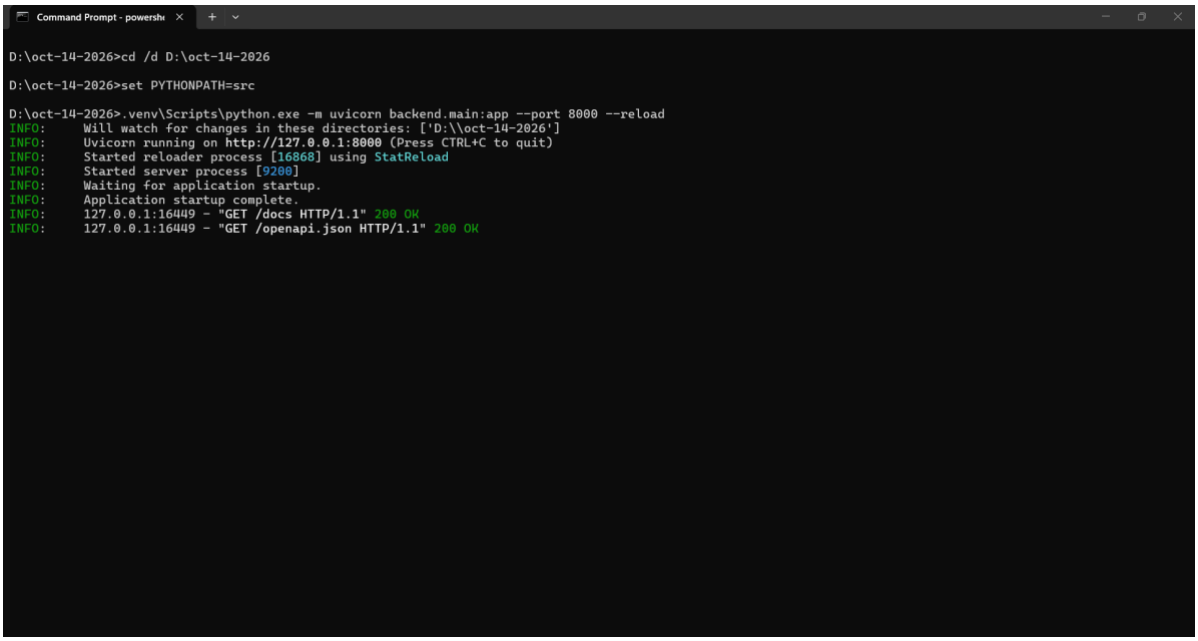


Figure 4.1: PoC backend started with uvicorn; /docs and /openapi.json return 200 OK

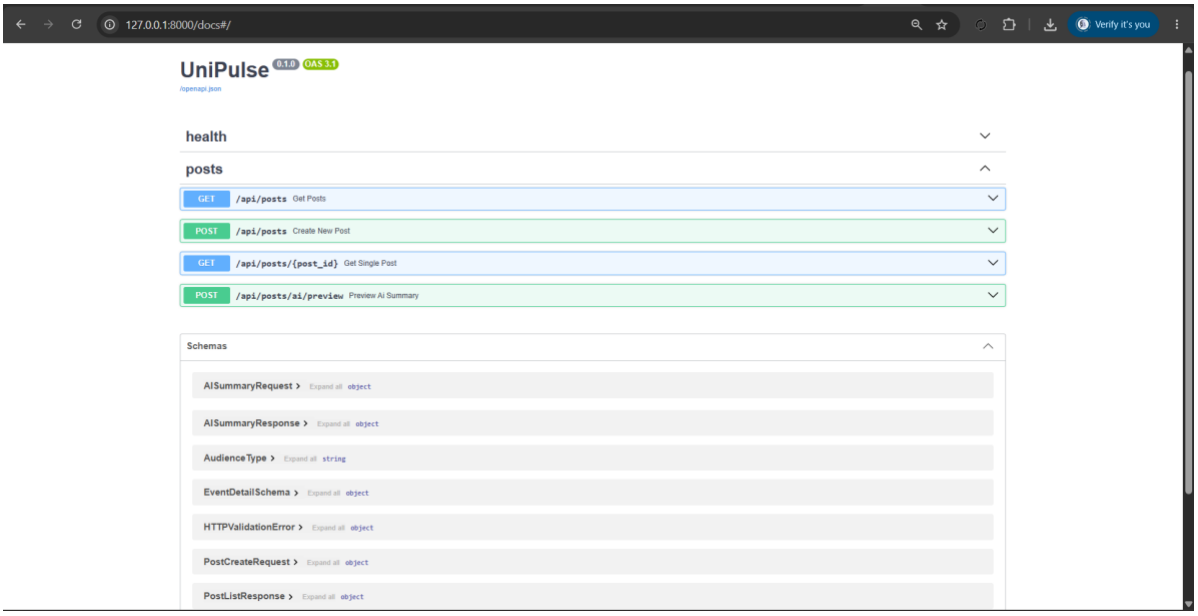


Figure 4.2: OpenAPI documentation generated by FastAPI from the PoC contract (http://127.0.0.1:8000/docs)

Demo Video Link: **https://youtu.be/jcd6ILOi890**

4.4 Three-Minute Demonstration Script

The team conducts a live 3-minute vertical slice demonstration during the oral defense examination:

• Minute 1 (Architecture & System Context): Introduce UniPulse, state the campus information fragmentation problem, and present the C4 Level 1 and Level 2 diagrams highlighting container boundaries and stakeholder roles.

• Minute 2 (Live Contract Execution & PulseDigest AI): In the browser, fill out the announcement form with 'ACM Study Jam', select the 'Campus Only' audience tier, and click 'AI Preview' to demonstrate PulseDigest event extraction (displaying location and date) and 2-sentence summary. Check 'Accept Summary' to confirm adherence to FR-AI-02 ('AI drafts, human verifies'). Click 'Publish Post' to trigger POST /api/posts (201 Created) and observe the dynamic rendering of the announcement card in the campus feed.

• Minute 3 (Server-Side Authorization & Test Verification): Demonstrate real implementation of authorizations on the server: Switch to the 'Student' mock role selector in the header. Try to access a Faculty Only Announcement via GET /api/posts/{faculty_post_id}. Ensure that the endpoint responds with a 403 Forbidden response containing a FORBIDDEN_AUDIENCE error code to indicate that audience privacy is applied at the backend query layer, and not just at the UI. Switch to 'Faculty' role to show access succeeds with 200 OK. Conclude by displaying the terminal running 22 passing pytest tests and clean Ruff linter verification.

Section 5: References

[1] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2016.

[2] C. Eden and F. Ackermann, Making Strategy: The Journey of Strategic Management. London, UK: SAGE Publications, 1998.

[3] ISO/IEC/IEEE, "Systems and software engineering — Life cycle processes — Requirements engineering," ISO/IEC/IEEE 29148:2018, 2018.

[4] AI1220 Course Staff, "AI Practices for Software Engineering: Principles, Verification and Ethics," Lecture 02, Week 05, 2026.

[5] A. Biryukov, D. Dinu, and A. Khovratovich, "Argon2: New Generation Password Hashing," in Proc. IEEE EuroS&P, 2016; IETF RFC 9106.

[6] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, Jun. 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[7] D. North, "Introducing BDD," Better Software, Mar. 2006.

[8] M. Cohn, User Stories Applied: For Agile Software Development. Boston, MA, USA: Addison-Wesley Professional, 2004.

[9] L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2021.

[10] F. Pizzati, "Architecture Design 1 & 2: Modularity, Quality Attributes, and Separation of Concerns," AI1220 Lecture Slides, Week 04, 2026.

[11] S. Brown, Visualising Software Architecture: A Practical Guide Using the C4 Model. Leanpub, 2018.

[12] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, Irvine, CA, USA, 2000.

[13] I. Fette and A. Melnikov, "The WebSocket Protocol," Internet Engineering Task Force (IETF), RFC 6455, Dec. 2011.

[14] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," ACM Transactions on Database Systems (TODS), vol. 6, no. 2, pp. 213–226, Jun. 1981.

[15] F. P. Brooks Jr., The Mythical Man-Month: Essays on Software Engineering, Anniversary ed. Reading, MA, USA: Addison-Wesley, 1995.

[16] K. Schwaber and J. Sutherland, "The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game," Scrum.org, Nov. 2020.

[17] D. Clegg and R. Barker, CASE Method Fast-Track: A Rad Approach. Wokingham, UK: Addison-Wesley, 1994.

[18] International Software Testing Qualifications Board (ISTQB), "Certified Tester Foundation Level Syllabus," Version 4.0, 2023.

[19] G. van Rossum, B. Warsaw, and N. Coghlan, "PEP 8 – Style Guide for Python Code," Python Enhancement Proposals, PEP 8, 2001. [Online]. Available: https://peps.python.org/pep-0008/

[20] Astral Software, "uv: An extremely fast Python package and project manager," 2024. [Online]. Available: https://github.com/astral-sh/uv